CSI201 Introduction to Networking

Coursework I Specification
Networked Quiz Tournament

Contribution to Overall Module Mark	**20%**
Submission Deadline	**10 Dec 2026 23:59**
Assessment Type	**Team coursework**
Learning Outcomes	**[A] [B] [D]**

Purpose and assessment scope

Each team must design, implement, document and test a command-line networked quiz tournament using Python 3.12 and raw TCP sockets. The submission consists of one server program, one client program, a local JSON question bank, supporting source modules, tests, and a complete development document.

No starter source code, client implementation, server implementation, or application-layer protocol is provided. The team owns the design of both programs and their communication protocol. The separate Application Layer Protocol Reference is informative only and may be adopted, modified, or ignored.

Non-negotiable interface and environment rules

The assessed product is a command-line network program. The following rules are conditions of assessment rather than optional design advice.

- **GUI prohibition:** The client and server must use a command-line interface only. A submission that includes, launches, embeds, or depends on any graphical user interface receives 0 marks for Part I. This includes desktop windows, web interfaces, browser-based control panels, and graphical dashboards.
- **Official environment:** All development verification, demonstrations, automated checks, and marking are performed only in the examiner-provided Docker image. Results obtained in another operating system or environment are not accepted as evidence that the submission works.
- **Paired assessment:** The examiner tests the submitted client with the submitted server. The examiner does not replace either side with an examiner implementation.
- **No installation step:** The programs must run with Python 3.12 Standard Library modules already available in the image. The submission must not download or install packages during execution.

Required runtime architecture

The submitted client and submitted server are assessed together. All components run inside the examiner-

provided Docker image. Only the server process may open the local question-bank file.

CLI client process(es)	↔	TCP/IP	↔	CLI server process	→	Local questions.json
User input and display		Team-designed protocol		Rooms, timing and scoring		Server read-only input

The arrows between the client and server are bidirectional because both requests and server-initiated updates may cross the connection. The arrow to questions.json is one-way because clients must not read the file directly.

Official Docker environment

The module team will publish an examiner Docker image and image tag on Learning Mall. Teams must use that image for every submitted test result and demonstration. Marking is performed in the same image. The image contains Python 3.12 and the Python Standard Library; network access for package installation must not be assumed.

Compulsory launch commands

```
python3 server.py --host <server_IP> --port <port> --questions <json_path>
python3 client.py --server <server_IP> --port <port> --id <user_id> --name <display_name>
```

- The server must print SERVER_READY after successful validation and after it begins listening.
- The server must bind to a user-selected port. Port numbers must not be hard-coded.
- The client and server must use standard input, standard output and standard error. Terminal cursor control is optional, but the program must remain usable in a basic non-graphical terminal.
- Paths must be relative or supplied through arguments. The programs must not rely on files outside the mounted submission directory.
- No database, background service, browser, desktop environment or external Internet service may be required.

Required functional behaviour

Task 1 Server start-up and validation

The server accepts the required arguments, validates the complete question bank, reports invalid input clearly, listens only after successful validation, and shuts down cleanly.

Task 2 Client identity and connection

Multiple client processes connect to the submitted server. Each active user has a unique user ID and display name. Duplicate active IDs must be rejected or resolved deterministically and documented.

Task 3 Quiz catalogue and rooms

A connected user can view available quizzes, create a room for a selected quiz, list joinable rooms, join a room, inspect participants, indicate readiness, leave, and start when the team's documented rules permit.

Task 4 Tournament play

The server controls question order and timing, distributes questions, accepts at most one answer per active player per question, closes questions, reveals results at the correct time, updates scores, and publishes a final ordered leaderboard.

Task 5 Concurrent operation

At least ten clients can remain connected and at least two rooms can run independently. Events, answers, timers and scores from one room must not affect another room.

Task 6 Failure handling

A malformed application message, invalid command, late or duplicate answer, missing client, unexpected disconnect, or Ctrl+C event must not corrupt unrelated sessions or permanently block the server.

Team-designed application-layer protocol

The exact framing method, message names, message fields, request identifiers, session representation, event model and error model are design decisions for the team. The protocol must operate over raw TCP sockets, must support the required paired workflow, and must be fully documented in the Development Documentation.

- The receiver must handle TCP fragmentation and coalescing correctly; one recv call must not be assumed to equal one application message.
- The protocol must distinguish normal responses, asynchronous server updates and errors in a documented way.
- The protocol must not disclose correct answers before the relevant question closes.
- The client and server are marked as a pair. Conformance to the informative reference protocol is not assessed unless the team chooses to claim it as its own design.

Normative question-bank input format

The question-bank format in this section is compulsory and is the only prescribed application-data format. The examiner may replace the submitted development file with another valid file that follows this schema. The server must validate the complete file before it opens its listening socket.

File-level requirements

- The file must be UTF-8 encoded JSON and must contain one top-level JSON object.
- The maximum accepted file size is 1 MB. An empty file or a larger file is invalid.
- The top-level object must contain schema_version and quizzes.
- Array order is significant. Quizzes and questions must be presented to players in file order unless the team clearly documents an intentional randomisation feature.
- Additional object members are permitted for forward compatibility and must not cause an otherwise valid file to be rejected, except that options must contain exactly A, B, C and D.
- If any required value is missing, has the wrong type, violates a stated range, or duplicates a required identifier, the complete file is invalid. The server must print a meaningful diagnostic, return a non-zero exit

status, and not listen for clients.

Required schema

Field	JSON type	Constraint	Invalidates
schema_version	integer	Exactly 1	Complete file
quizzes	array	1 to 20 quiz objects	Complete file
quiz_id	string	1-32 characters; letters, digits, underscore or hyphen; unique	Complete file
title	string	1-100 visible characters	Containing quiz
description	string	0-300 visible characters	Containing quiz
questions	array	5-50 question objects	Containing quiz
question_id	string	1-32 characters; same character set; unique in file	Complete file
text	string	1-500 visible characters	Containing question
options	object	Exactly keys A, B, C and D; each value 1-200 characters	Containing question
answer	string	Exactly A, B, C or D	Containing question
time_limit	integer	5-60 seconds	Containing question
points	integer	1-1000 points	Containing question

Minimal valid example

The following file is valid and contains one quiz with the minimum five questions. Correct answers remain server-side and must not be disclosed before the corresponding question closes.

```
{
  "schema_version": 1,
  "quizzes": [
    {
      "quiz_id": "network-basics",
      "title": "Networking Fundamentals",
      "description": "Five introductory networking questions",
      "questions": [
        {
          "question_id": "NET001",
          "text": "Which transport protocol provides reliable byte-stream delivery?",
          "options": {"A":"UDP", "B":"TCP", "C":"ARP", "D":"ICMP"},
```

```
          "answer": "B", "time_limit": 15, "points": 100
        },
        {
          "question_id": "NET002",
          "text": "Which device forwards packets between IP networks?",
          "options": {"A":"Router", "B":"Hub", "C":"Repeater", "D":"Patch panel"},
          "answer": "A", "time_limit": 15, "points": 100
        },
        {
          "question_id": "NET003",
          "text": "Which protocol maps a domain name to an IP address?",
          "options": {"A":"DHCP", "B":"HTTP", "C":"DNS", "D":"FTP"},
          "answer": "C", "time_limit": 15, "points": 100
        },
        {
          "question_id": "NET004",
          "text": "How many bits are in an IPv4 address?",
          "options": {"A":"16", "B":"32", "C":"64", "D":"128"},
          "answer": "B", "time_limit": 15, "points": 100
        },
        {
          "question_id": "NET005",
          "text": "Which layer provides end-to-end process communication?",
          "options": {"A":"Physical", "B":"Data link", "C":"Network",
"D":"Transport"},
          "answer": "D", "time_limit": 15, "points": 100
        }
      ]
    }
  ]
}
```

Required handling behaviour

- Only the server reads questions.json. A client must obtain quiz and question information through its connection to the submitted server.
- Public quiz listings may include quiz_id, title, description and question count, but must not reveal question text, options or answers before play.
- During a question, the client may receive question_id, text, options, time_limit and points. The answer must remain confidential until the question closes.
- The server must use the points value for a correct accepted answer. An incorrect, late, duplicate or missing answer receives zero points unless the team's documented game design applies an additional non-negative rule consistently.
- The server must use each time_limit as the authoritative limit for the corresponding question.

Permitted Python libraries

Only Python 3.12 Standard Library modules are permitted. No third-party package is required or allowed. Suitable modules include argparse, collections, concurrent.futures, contextlib, dataclasses, enum, json, logging, os, pathlib, queue, random, secrets, select, selectors, signal, socket, statistics, string, struct, sys, tempfile, threading, time, typing, unittest and uuid.

The network implementation must directly use socket. High-level servers and frameworks, including Flask, Django, FastAPI, aiohttp, Requests, WebSocket libraries, Socket.IO, gRPC, ZeroMQ, Twisted and socketserver, are not permitted. Use of asyncio high-level server APIs is not permitted for the assessed network core.

Development documentation

Each team must submit one complete software-development document. It is assessed as an engineering record of the system rather than as a short academic essay. The document must allow a technically competent reader to understand the requirements, reproduce the deployment, maintain the design, and evaluate the evidence.

The recommended length is 12 to 18 pages, excluding the cover page, table of contents, references, and appendices. The Development Documentation must use the single-column report format adopted for both Coursework I and Coursework II, with A4 pages, 11 pt readable body text, numbered headings, page numbers, consistent captions, and IEEE referencing style.

Required section	Minimum content
Document control and executive summary	Title, version, date, team members, student IDs, revision history, purpose, product scope, principal outcome and known limitations.
Requirements analysis	Stakeholders, assumptions, constraints, functional requirements, non-functional requirements, acceptance criteria and a requirements traceability matrix using stable requirement IDs.
System architecture	Context, component and deployment views showing client processes, server process, connections, question-bank access, responsibilities, interfaces and trust boundaries.
Detailed software design	Module responsibilities, key classes or data structures, room and quiz state models, concurrency model, synchronization decisions, timing model, error strategy and shutdown design.
Data design	The compulsory question-bank schema, in-memory entities, ownership of mutable state, validation rules, confidentiality of answers and lifecycle of session, room and score data.
Application-layer protocol design	The team's actual framing method, encoding, message catalogue, field definitions, request and event semantics, sequence diagrams, error handling, compatibility assumptions and examples. The separate reference appendix may be used for ideas but does not replace this section.

Required section	Minimum content
Implementation record	Development environment, source layout, important algorithms, use of Python Standard Library modules, implementation trade-offs, difficult issues, and how the implementation corresponds to the design.
Deployment and operation	Exact commands for the examiner Docker image, configuration and arguments, expected start-up output, normal shutdown, log locations, resource assumptions and a concise operator guide.
Verification and validation	Test strategy, test environment, requirement-to-test traceability, repeatable test cases, expected and actual results, malformed-input tests, timing tests, concurrency tests, defect records and selected terminal evidence.
Reliability and security	Input validation, answer confidentiality, race-condition prevention, client disconnects, denial-of-service considerations within scope, sensitive logging, failure recovery and residual risks.
Project management and contribution	Work breakdown, milestones, repository evidence, review practice, individual responsibilities, contribution percentages totalling 100%, and any changes from the original plan.
Limitations, future work and references	Evidence-based limitations, realistic improvements, conclusion, and consistent citations for external technical material.

Documentation evidence rules

- A screenshot is supporting evidence, not a substitute for an explanation, test procedure, or result table.
- Do not paste the complete source code into the document. Include only short extracts when they are necessary to explain a design decision.
- Every claimed requirement must map to a design element and at least one verification activity.
- All commands and results must come from the examiner-provided Docker image.
- Statistical graphs and plotting libraries are not required. Engineering diagrams, state models, sequence diagrams, tables and terminal captures are expected where they improve clarity.

Submission

The team leader submits one ZIP archive through Learning Mall. The archive name and deadline will be confirmed on Learning Mall.

Required archive structure

```
CSI201-CW-Part-I-TeamName.zip
├── Codes
│   ├── server.py
│   ├── client.py
│   ├── questions.json
│   ├── README.md
```

```
│   ├── tests
│   └── additional Python modules if required
└── Documentation
    └── Development_Documentation.pdf
```

- Do not include a virtual environment, package cache, IDE folder, executable binary, database server, Docker image, Dockerfile, or unrelated data.
- README.md must contain the exact start-up commands, CLI command summary, assumptions and known limitations.
- The supplied examiner Docker image, not a team-created image, is used for all assessed execution.
- The submitted questions.json must be valid and contain at least two quizzes with at least five questions per quiz.

Video Demonstration:

Each team should submit a 10-minute video demonstration showing the implementation and operation of Tasks 1–5. Every team member must participate in the video and explain/demonstrate a meaningful part of the team's implementation, demonstrating their understanding of the submitted work.

Late Submission Policy:

The University applies penalties to the late submission of assessed coursework. Students are responsible for ensuring that the complete coursework is submitted through **Learning Mall before the stated deadline**. Late submissions will be handled in accordance with the **University's regulations for late submission of assessed coursework**.

Artificial Intelligence Permissions: Permitted with Restrictions

Students may use generative AI tools to support their understanding of the coursework requirements and to assist with programming and software development activities. This may include using AI to explain concepts, provide guidance on programming approaches, assist with debugging, or suggest code. However, students remain fully responsible for all material submitted for assessment. **Every group member must understand, critically evaluate, test, and verify any AI-assisted or AI-generated content used in the coursework.** Each group member must also be able to explain in detail the submitted implementation, code, design decisions, results, and analysis when requested. The use of AI-generated content does not remove the student's responsibility for the correctness, originality, or academic integrity of the submitted work.

The use of generative AI to create images, diagrams, plots, figures, screenshots, or other visual evidence for submission is not permitted. All submitted figures and visual evidence must be produced directly from the students' own implementation, experimental results, analysis, or appropriate conventional software tools.

Students must comply with the University's requirements for **acknowledging and appropriately using generative AI in assessed work by referencing the AI tool used**.

Academic Integrity:

Students are expected to comply with the University's regulations on **academic integrity**. Any violation of academic integrity, including **plagiarism, collusion, copying, submission of commissioned or procured work, academic offenses involving Artificial Intelligence (AI), and falsification or fabrication of data**, may

result in penalties and demerit points in accordance with University regulations. Please refer to the **University's Academic Integrity Policy on e-Bridge** for guidance.

Although this is a team assessment, collaboration is permitted **only among members of the same registered team**. Students must not share their implementation, code, report, experimental results, or other assessed material with members of other teams, nor use material produced by another team. All submitted experimental results, screenshots, figures, CSV files, and other evidence must be generated from the team's **own implementation and experiments**. Fabricating, modifying, or presenting results that were not obtained from the team's actual implementation constitutes an academic integrity violation.

Mark allocation

Part I is marked out of 100. Development Documentation contributes 60 marks and System Implementation contributes 40 marks.

Development Documentation 60 marks

Category	Assessment focus	Marks
Document control and executive summary	Version, scope, outcome and limitations	4
Requirements analysis and traceability	Functional and non-functional requirements, acceptance criteria and traceability	8
Architecture and detailed design	Context, components, deployment, modules, state, data and responsibilities	12
Protocol, timing and concurrency design	Actual team protocol, sequences, framing, errors, synchronization and timing	10
Implementation and Docker operation	Source organization, decisions, algorithms and reproducible operation	8
Verification and validation	Test plan, traceability, repeatable evidence, results and defect analysis	10
Reliability, security and limitations	Validation, confidentiality, failures, residual risks and future work	4
Project management and contribution	Milestones, repository evidence, roles and percentages	2
Writing, structure and references	Professional organization, captions, accuracy and consistent citation	2
Total		60

Development Documentation performance descriptors

The following descriptors are applied proportionally to each Development Documentation category.

Band	Percentage of available marks	Descriptor
Outstanding	85-100%	Comprehensive, accurate, well justified, and supported by strong traceability and reproducible evidence.
Excellent	70-84%	Complete and accurate, with clear justification and evidence; only minor weaknesses are present.

Good	60-69%	Mostly complete and correct, with adequate explanation and evidence but some gaps or limited analysis.
Satisfactory	50-59%	Addresses the main requirements but is mainly descriptive, with limited justification, evaluation, or evidence.
Limited pass	40-49%	Partially addresses the requirements, with notable omissions, inaccuracies, or weak evidence.
Insufficient	0-39%	Substantially incomplete, inaccurate, inconsistent with the submitted system, or unsupported by assessable evidence.

System Implementation 40 marks

Category	Assessment focus	Marks
Docker start-up, CLI and question bank	Required commands, validation, server-only file access, no external dependencies	6
Complete paired quiz workflow	Submitted client and server complete login, room, quiz, answer and result workflow	12
Multi-client room management	Create, join, readiness, host behaviour, leave and room lifecycle	7
Quiz correctness, timing and scoring	Question order, deadlines, accepted answers, confidentiality and leaderboard	6
Concurrency and isolation	At least ten connected clients, two independent rooms and protected shared state	5
Robustness and shutdown	Malformed input, disconnects, resource cleanup and Ctrl+C behaviour	4
Total		40

System Implementation performance descriptors

The following descriptors are applied proportionally to each System Implementation category. Marks are based on behaviour demonstrated in the examiner-provided Docker image.

Band	Percentage of available marks	Descriptor
Outstanding	85-100%	All required behaviour works correctly and reliably under normal and adverse conditions.
Excellent	70-84%	Almost all required behaviour works reliably, with only minor defects or omissions.
Good	60-69%	The core workflow works, but some secondary functions or edge cases are incomplete.
Satisfactory	50-59%	A basic end-to-end workflow is demonstrated, but notable functions are missing or unreliable.
Limited pass	40-49%	Only part of the required workflow operates and substantial intervention may be required.

Insufficient	0-39%	Little or no required behaviour can be demonstrated reliably.

Zero-mark and unassessable conditions

- Any GUI in the submitted client or server results in 0 marks for Part I.
- If the submitted paired system cannot be started in the examiner-provided Docker image using the documented commands, System Implementation receives 0 marks. The readable Development Documentation may still be marked.
- A missing Development_Documentation.pdf receives 0 for Development Documentation.

Paired marking procedure

The examiner uses the submitted server and submitted client together inside the official Docker image. The examiner does not use a private client to test the server and does not use a private server to test the client. Terminal wording may vary except for the required SERVER_READY signal, but the complete behaviour must be demonstrable through the submitted client.

1. Validate archive structure and start the submitted server with a valid hidden question bank.
2. Repeat server start-up with missing, malformed and schema-invalid question banks.
3. Start multiple submitted clients and verify identity handling and normal exit.
4. List quizzes, create and join rooms, inspect participants, apply readiness rules and start a quiz.
5. Complete a quiz with correct, incorrect, duplicate, missing and late answers; verify timing and leaderboard behaviour.
6. Operate at least ten clients across at least two rooms and verify isolation.
7. Disconnect clients unexpectedly, send malformed or invalid input, and confirm unaffected clients and rooms continue.
8. Stop the server with Ctrl+C and verify that listening and connection resources are released.

Team contribution and authorship

Every member must contribute and must be able to explain the submitted requirements, architecture, protocol, buffering, state transitions, concurrency, synchronization, timing, question-bank validation, testing and error handling. Contribution percentages in the Development Documentation must sum to 100%. Repository history, demonstrations and interviews may be used to review the declaration.

Submitted work remains subject to the University's academic integrity regulations.

Individual contribution adjustment

All members normally receive the group mark. Where the evidence confirms a substantial difference in contribution, the individual mark is calculated as:

Individual mark = Group mark × (individual contribution percentage / highest contribution percentage)

The member with the highest verified contribution receives the full group mark. Other members receive proportionally adjusted marks, rounded to the nearest whole number.

For example, for a group mark of 72 and contributions of 30%, 30%, 25%, and 15%, the individual marks are 72, 72, 60, and 36.

Contribution percentages may be verified using repository history, documented responsibilities, report authorship, test records, the video demonstration, and interviews. If a substantial difference cannot be verified, all members receive the group mark.

Video Demonstration

Each team must submit a video of no more than 10 minutes demonstrating Tasks 1-5 in the examiner-provided Docker environment. Every member must explain or demonstrate a meaningful part of the submitted work.

The video does not receive a separate mark. It is supporting evidence for the Development Documentation, and its production or editing quality is not assessed. It does not replace any required explanation, test result, or evaluation in the document.

The examiner may use the video to verify whether claims and evidence in the Development Documentation are credible and consistent with the submitted system. Unsupported or contradictory claims may receive reduced or no credit in the affected documentation category. System Implementation marks remain based on behaviour reproduced by the examiner.